Teste 02 — RNN, LSTM e GRU

Respostas (versão direta) — Aluno: Bruno Araujo

---

1. Dataset

Questão 1

A entrada é cada avaliação de filme transformada numa lista de números, em que cada número representa uma palavra. Todas as listas têm 120 posições. O rótulo **0** quer dizer avaliação **negativa** e o **1** quer dizer **positiva**.

A ordem importa porque muda o sentido: "*good*" é positivo, mas "*not good*" é negativo. Em "*achei que seria ruim, mas foi ótimo*", o que vale é o final. Por isso usamos redes recorrentes, que leem a frase palavra por palavra e lembram do que veio antes.

Questão 2

Conjunto	Total	Negativas	Positivas
Treino	10.000	5.000	5.000
Validação	2.000	1.000	1.000
Teste	2.000	1.000	1.000

Os três modelos usam os mesmos dados para que a comparação seja justa: se um modelo for melhor, é por causa da arquitetura, e não porque recebeu exemplos mais fáceis.

---

2. Sequências e embedding

Questão 3

- `X_train (10000, 120)` : 10.000 avaliações com 120 palavras cada.
- `vetores_demo (2, 120, 24)` : 2 avaliações, 120 palavras, e cada palavra virou um vetor de 24 números.

O que cada valor significa:

- **T = 120:** quantas palavras a rede lê (passos de tempo).
- **F = 24:** quantos números representam cada palavra.
- **H = 16:** quantos neurônios a camada recorrente tem, ou seja, o tamanho da "memória".

Questão 4

- **Curto:** tinha 4 palavras e foi completado com zeros no final até chegar a 120 (**padding**).
- **Longo:** tinha 130 palavras e as 10 primeiras foram cortadas (**truncamento**).

Com `truncating="pre"`, o que se perde é o **começo** da avaliação. Como a avaliação típica tem 176 palavras, mais da metade perde o início.

Questão 5

Os IDs são só números de identificação e não dizem nada sobre o significado da palavra. O **Embedding** transforma cada ID num vetor que a rede aprende durante o treino, de modo que palavras de sentido parecido (como "*great*" e "*excellent*") ficam com vetores parecidos.

Validação e teste não podem ajustar os pesos porque servem para medir o modelo em dados que ele nunca viu. Se fossem usados no treino, o resultado ficaria artificialmente bom (vazamento de dados).

---

3. Estados das redes

Questão 6

`(2, 120, 16)` = 2 avaliações × 120 passos × 16 valores de memória em cada passo. A saída tem os 120 passos porque usamos `return_sequences=True`.

Os pesos **não mudam** de um passo para outro: a rede usa os mesmos pesos em todas as palavras.

Questão 7

- **RNN:** passa adiante só o estado `h`.
- **LSTM:** passa `h` e `c`.
- **GRU:** passa só `h`.

Na LSTM, o `c` é a memória de longo prazo, que guarda informação por muitos passos. O `h` é a memória de curto prazo, a parte da memória que a rede "mostra" em cada passo e envia adiante.

---

4. Cálculos das portas

Questão 8 — LSTM

Fórmula: **c = f × c_anterior + i × g**

Neurônio	O que manteve (f × c_ant.)	O que entrou (i × g)	Novo c
1	0,25 × 0,8 = 0,200	0,60 × (−0,5) = −0,300	**−0,100**
2	0,90 × (−0,4) = −0,360	0,20 × 0,7 = 0,140	**−0,220**

c = [−0,100; −0,220]

O neurônio 1 esqueceu quase tudo e aceitou a informação nova. O neurônio 2 manteve quase toda a memória antiga.

Questão 9

- **f (esquecimento):** decide quanto da memória antiga manter.
- **i (entrada):** decide quanto da informação nova guardar.
- **o (saída):** decide quanto da memória mostrar como saída.

O **g não é uma porta**. As portas são "torneiras" com valor entre 0 e 1 que controlam o quanto passa. O `g` é o próprio conteúdo novo, com valor entre −1 e 1.

Questão 10 — GRU

Fórmula: **h = z × h_anterior + (1 − z) × h_candidato**

Neurônio	Parte antiga (z × h_ant.)	Parte nova ((1 − z) × h_cand.)	Novo h
1	0,80 × 0,60 = 0,480	0,20 × (−0,20) = −0,040	**0,440**
2	0,25 × (−0,40) = −0,100	0,75 × 0,50 = 0,375	**0,275**

h = [0,440; 0,275]

- **z (atualização):** decide no final quanto manter do estado antigo e quanto trocar pelo novo.
- **r (reset):** atua antes, decidindo quanto do passado usar para criar o candidato.

---

5. Parâmetros

Questão 11

RNN: entrada 24 × 16 = 384, recorrente 16 × 16 = 256, bias 16. **Total = 656.**

LSTM: tem 4 blocos iguais ao da RNN (f, i, o, g): 4 × 656 = **2.624**. Isso dá 1.536 de entrada, 1.024 recorrentes e 64 de bias.

Os dois valores batem com o código. Se `T` fosse de 120 para 240, **nada mudaria**, porque os mesmos pesos são usados em todas as palavras.

Questão 12

GRU: 3 blocos, com 3 × (384 + 256) = 1.920 pesos, mais 2 × 48 = 96 de bias. **Total = 2.016.**

São 3 blocos porque, além das duas portas (z e r), o candidato também tem seus pesos. O Keras usa **dois biases por bloco** na GRU (um para a entrada e outro para a parte recorrente), enquanto a LSTM usa um só. Mesmo assim, a GRU é menor que a LSTM.

---

6. Treinamento

Questão 13

- **Hiperparâmetros de arquitetura:** tamanho do vocabulário (5.000), T = 120, F = 24, H = 16, tipo da camada (RNN, LSTM ou GRU) e camada de saída sigmoid.
- **Hiperparâmetros de treinamento:** otimizador Adam, taxa 0,001, lote de 128, no máximo 8 épocas e Early Stopping com paciência 2.
- **Aprendido pela rede:** pesos do embedding, da camada recorrente e da camada de saída.

O número de épocas variou porque o Early Stopping para cada modelo 2 épocas depois de a validação parar de melhorar. A RNN parou na época 5, e a LSTM e a GRU na época 4.

Questão 14

Modelo	Melhor época	Menor val_loss
RNN	3	0,4206
LSTM	2	**0,3838**
GRU	2	0,4006

A **LSTM** teve a menor loss de validação.

Os três modelos mostram **overfitting**: a loss de treino continua caindo, mas a de validação volta a subir depois da melhor época (na LSTM, de 0,38 para 0,50). Ou seja, a rede começa a decorar o treino. O Early Stopping resolveu isso ao recuperar os pesos da melhor época.

---

7. Teste

Questão 15

Modelo	Acurácia no teste
RNN	82,5%
LSTM	**83,5%**
GRU	83,4%

Os três ficaram bem acima dos 50% de quem "chuta" sempre a mesma classe. A LSTM foi a melhor, mas por muito pouco (0,1 ponto sobre a GRU).

Isso não prova que a LSTM é sempre melhor: foi um único teste, com uma única semente, e a diferença é de poucas avaliações. Seria preciso repetir várias vezes para ter certeza.

Questão 16

- **Ex. 5 (positiva):** os três modelos deram cerca de 0,87–0,89 e todos acertaram.
- **Ex. 7 (positiva):** a RNN deu 0,174 e errou com bastante confiança (achou que era negativa). A LSTM (0,750) e a GRU (0,643) acertaram.

Probabilidade alta não garante acerto. A probabilidade mostra o quanto o modelo está confiante, não se ele está certo, e a RNN no Ex. 7 é a prova disso.